

## 四、ACI 设计哲学：在百万行代码中精准定位与安全修改

核心问题：Agent 如何在大规模代码库中高效定位目标、安全修改代码？

OODA 定位：Observe 环节（感知质量）/ 重点覆盖 Localization 阶段

### 从上一节的问题切入

上一节结尾留了一个问题：

Agent 在 Localization 阶段如何高效感知代码库？在百万行代码中，如何用“刚好够”的信息量找到目标，而不是把整个文件塞进上下文？

上一节讲了 Orient——Agent 收到报错信息后如何理解和推理，决定了它是能纠错还是陷入死循环。但 Orient 的质量上限取决于它前面那个环节：**Observe（感知）**。

回到 Boyd 推论二（第一讲 § 1.6）：

感知设计决定循环质量的上限。F-86 的气泡座舱 = Agent 的工具接口设计。如果 Agent 每次只能看到噪声满满的全文输出，它的 Orient 再强也无济于事——垃圾进、垃圾出。

这一节的核心矛盾：

上下文窗口有限 vs 代码库巨大

- 信息过多 = 噪声淹没关键信号（token 浪费，Orient 被干扰）
- 信息过少 = 盲人摸象（定位失败，根本找不到修改点）
- Observe 的核心挑战 = 用“刚好够”的粒度获取信息

上一节的 Demo 中，30B MoE 模型成功修复了 `buggy_calc.py`——但那个文件只有 39 行。Agent 一次 Read 就能看到全貌，Localization 几乎不存在。现在想象一个真实场景：一个 Python 项目有 100 个文件、数万行代码，模型**甚至找不到正确的文件**——Orient 再强也救不了 Observe 的失败。AutoCodeRover 论文的失败案例分析（Section 6.3）证实了这一点：**18% 的失败是因为定位到了错误的文件**，连正确的战场都没找到。

三篇论文分别给出了三种不同的 Observe 策略：

| 论文            | 核心策略        | 信息控制方式         |
|---------------|-------------|----------------|
| SWE-agent     | 定制工具接口（ACI） | 100 行窗口 + 搜索命令 |
| AutoCodeRover | AST 结构化检索   | 语义单元（类/方法级）    |
| Agentless     | 三层漏斗定位      | 骨架 → 全文层次递进    |

## 4.1 为什么传统的检索在编程中常常失效

在进入三篇论文之前，先理解一个背景：为什么不能直接用检索？

SWE-bench 论文定义的 RAG 基线使用 **BM25** 检索最相关的代码文件，然后让模型直接生成 patch。SWE-agent 论文（Table 1）的对比数据非常有说明力：

| 方法                                | SWE-bench Lite |
|-----------------------------------|----------------|
| RAG + GPT-4 Turbo（BM25 检索）        | 2.67%          |
| RAG + Claude 3 Opus（BM25 检索）      | 4.33%          |
| SWE-agent + GPT-4 Turbo（Agent 交互） | <b>18.00%</b>  |

BM25 检索 + 一次性生成的效果极差。为什么？

### 词汇鸿沟（Vocabulary Gap）

用户报告一个 bug：“当输入偶数个成绩时，中位数计算结果不对”。

你用这句话做 BM25 检索，期望找到 `buggy_calc.py` 中的 `median` 计算逻辑。但实际代码长这样：

```
sorted_scores = sorted(scores)
mid = n // 2
if n % 2 == 0:
    median = sorted_scores[mid]    # bug 在这里
```

代码中没有“偶数”、“中位数计算结果不对”这些词。有的是 `sorted_scores`、`mid`、`n % 2`——纯符号操作。BM25 依赖**词汇重叠**，Issue 描述和代码之间的词汇鸿沟太大了。

那向量检索 (embedding) 呢？**Agentless** 是唯一同时使用了 prompting + embedding 检索的论文（用 OpenAI `text-embedding-3-small`）。它的消融实验 (Table 2) 揭示了有趣的结论：

| 定位方法                  | 文件级命中率 |
|-----------------------|--------|
| 仅 Prompting（给文件树让模型选） | 78.67% |
| 仅 Embedding（向量相似度检索）  | 67.67% |
| 两者结合                  | 81.67% |

**Prompting（让模型“看”文件结构并推理）比 embedding 检索更准确。** Embedding 检索的贡献是补充——它能找到一些 prompting 遗漏的文件，但单独使用效果更差。这佐证了代码 Localization 的核心不是“找相似文本”，而是**理解 Issue 描述和代码结构之间的因果关系**——这需要推理，不是相似度匹配。

### 代码的结构性

代码不是散文。它有严格的结构：模块 → 类 → 方法 → 行。一个 bug 的定位路径往往是：

从 Issue 描述出发

→ 哪个模块负责这个功能？（文件级）

→ 模块中哪个类/函数处理这个逻辑？（结构级）

→ 函数中哪几行有问题？（行级）

这个**从粗到细的漏斗过程**，才是编程场景中有效的 Localization 策略——无论是人类开发者还是 AI Agent 都遵循这个模式。三篇论文分别用不同方式实现了这个漏斗。

## 4.2 SWE-agent：ACI 设计——给 Agent 造一个好用的驾驶舱

**SWE-agent** (Yang et al., Princeton, 2024) 的核心洞察不是让模型更聪明，而是让模型的工作界面更好用。

回到 Boyd 的 F-86 比喻：F-86 赢在气泡座舱（360° 视野）和液压飞控（轻推即响应），不是赢在更大的发动机。SWE-agent 将这个思路搬到了 AI 编程领域——设计一层介于模型和操作系统之间的工具接口，叫做 **ACI（Agent-Computer Interface）**。

## ACI 四原则

SWE-agent 论文（Section 3）总结了四条 ACI 设计原则：

### 原则一：简洁动作（Simple Actions）

每个命令做一件事，降低 action 空间复杂度。不是给模型一个万能的 `bash` 命令让它自己拼参数，而是提供专门的 `find_file`、`search_dir`、`search_file` 命令，每个命令的职责明确、参数简单。

### 原则二：紧凑动作（Compact Actions）

工具的输出要精简，不浪费 token。一个 `cat` 命令输出 500 行文件内容会吞噬大量上下文空间；SWE-agent 的 File Viewer 每次只显示 **100 行窗口**，需要看更多就用 `scroll_down`。

### 原则三：信息丰富的反馈（Informative Feedback）

错误信息要有诊断价值。比起 `Error: command failed`，更好的反馈是 `Error: file not found. Did you mean tests/buggy_calc.py?` ——错误信息本身就包含了修正的线索。

### 原则四：护栏（Guardrails）

防止常见错误。SWE-agent 的 `edit` 命令写入前会自动执行 **lint 语法检查**，如果写入的代码有语法错误就回滚，不让错误代码落盘。

## 搜索工具的三层粒度

SWE-agent 为 Localization 提供了三个粒度的搜索工具：

```
find_file "calc.py"           → 项目级：在整个项目中搜索文件名
search_dir "median" src/      → 目录级：在指定目录中搜索文本
search_file "variance" calc.py → 文件级：在指定文件中搜索文本
```

这三层粒度对应了从粗到细的定位漏斗：先找到文件，再找到相关代码段，最后锁定具体行。

## File Viewer：带窗口的文件浏览器

SWE-agent 不用 `cat`。它发明了一个**带窗口的文件浏览器**：

|              |                          |
|--------------|--------------------------|
| open calc.py | → 打开文件，显示第 1-100 行       |
| scroll_down  | → 显示第 101-200 行          |
| goto 50      | → 跳到第 50 行附近，显示第 1-100 行 |

每次只显示 100 行。模型需要“翻页”才能看到更多内容。这个设计的核心目的是**保护上下文窗口**——一个 2000 行的文件如果全部 cat 进去，后续几轮对话的推理质量都会受影响。

## 关键数据：ACI > 模型

SWE-agent 论文 Table 2 的消融实验是全文最有价值的发现：

|                               |            |
|-------------------------------|------------|
| Shell-only baseline (无 ACI) : | 1.7%       |
| + ACI 工具接口 :                  | 12.47%     |
| <hr/>                         |            |
| ACI 贡献 :                      | +10.7 个百分点 |

**改善工具接口设计带来的提升 (+10.7pp) 远大于换更好的模型。** 这是 Boyd 推论二的直接实证：在 AI 编程场景中，感知设计（Observe 的质量）对最终结果的影响，超过了模型能力本身。

对 MVP 的启示：与其一味追求更大的模型，不如优化工具的输出格式（限制输出长度、增加诊断信息、添加护栏）。

**CC 源码实证**（`src/tools/GrepTool/GrepTool.ts`）：CC 的 `Grep` 工具默认 `head_limit: 250`，超过时截断输出并提示“results truncated”。这与 SWE-agent 的“紧凑输出”原则完全一致——**工具输出越精简，留给后续推理的空间越大。**

**CC 源码实证**（`src/tools/FileReadTool/FileReadTool.ts`）：CC 的 `Read` 工具支持 `offset + limit` 参数，可以只读取文件的特定行范围。这是 SWE-agent File Viewer 窗口机制的工业级实现。系统提示中甚至明确指示模型：“By default, it reads up to 2000 lines... You can optionally specify a line offset and limit (especially handy for long files).”

但更关键的是 CC 的**大文件防护机制**（`FileReadTool/limits.ts`）——CC 不是靠提示词“建议”模型少读，而是用硬性门槛**拒绝**过大的读取：

| 防线          | 门槛            | 触发时机 | 行为                                            |
|-------------|---------------|------|-----------------------------------------------|
| 文件大小门槛      | 256 KB        | 读取前  | 抛出 <code>FileTooLargeError</code> ，拒绝读取       |
| 输出 token 门槛 | 25,000 tokens | 读取后  | 抛出 <code>MaxFileReadTokenExceededError</code> |

两层都返回**错误信息**（不是静默截断），错误中明确告诉模型“use offset and limit”。这与 ACI 原则三“信息丰富的反馈”一致——**错误本身就是教学信号**，迫使模型学习分页读取的行为模式。

我们的 MVP 复刻了这个设计：`ReadTool.MAX_LINES_WITHOUT_LIMIT = 250`（按 CC 的 2000 行与 200K context 的比例缩放到 16K context），超过时返回 Error + 前 20 行预览 + Grep 建议。错误信息中直接给出 Grep 命令示例（如 `Grep(pattern="validate|reject|error", path="<file>")`），引导模型用搜索替代分页读取。与 CC 一致，当模型指定了 `offset/limit` 时不限制单次读取行数——真正的上下文保护靠 Micro Compact 层。这是本节 Demo 中的关键改进——没有这个门槛，Agent 读入一个 625 行的 TypeScript 文件后，累积上下文超过模型的 `max_model_len`，导致 vLLM 拒绝生成，整个会话崩溃。

**CC 源码实证**（`query.ts`，`applyToolResultBudget()`）：CC 在每轮工具执行后会检查结果长度，过长的输出会被截断。这对应 ACI 的“紧凑动作”原则——从 harness 层面强制控制信息粒度，而不是依赖模型自觉。

CC 的上下文防线远不止于工具层面。它有一套**七层纵深防御体系**保护上下文窗口不溢出：

- 第 1 层：工具输出限制 — 单工具 50K chars，每轮总计 200K chars
- 第 2 层：Read token 门槛 — 文件读取 > 25K tokens 则拒绝（本节重点）
- 第 3 层：Snip Compaction — 删除旧消息
- 第 4 层：Micro Compact — 清除旧工具结果 → [Old tool result content cleared]
- 第 5 层：Context Collapse — 折叠冗余上下文视图
- 第 6 层：Auto Compact — 当 token 接近 contextWindow - 13K 时，用 LLM 调用摘要化整个对话历史
- 第 7 层：Reactive Compact — API 返回 prompt-too-long 错误时的最后防线

这七层的设计思路是**逐级收紧，越往后代价越大**：工具限制几乎零成本，Micro Compact 只替换文本，而 Auto Compact 需要额外一次 LLM 调用来生成摘要。CC 的工程目标是尽量在前几层就控制住信息量，避免触发昂贵的后几层。

MVP 实现了第 1-2 层（Read 门槛 + 错误处理）和第 4 层（Micro Compact）。MVP 的 `_microcompact()` 在每轮 API 调用前，将旧的 `tool_result` 内容替换为 `"[Old tool result content cleared]"`，只保留最近 3 条消息的完整工具结果。此外，MVP 还引入了**预算警告机制**：当 Agent 接近轮次上限时（倒数第 2 轮），注入一条“STOP searching and WRITE YOUR FINAL ANSWER”的提示，迫使模型从信息收集切换到信息综合。这是因为本地 30B MoE 模型缺乏自主的“停止搜索、开始回答”判断能力——不同于 Opus/Sonnet 级模型，它需要外部信号来触发模式切换。更深的 Auto Compact（需要额外 LLM 调用来摘要化对话）超出了教学 MVP 的范围，但理解这个纵深体系对于认识真实 Agent 系统的工程复杂度很重要。

### 4.3 AutoCodeRover：用代码结构导航——代码不是文本

**AutoCodeRover** (Zhang et al., NUS, 2024) 提出了一个不同的思路：**代码有结构，应该用结构化的方式搜索，而不是把它当纯文本处理。**

SWE-agent 的搜索是文本级的——`search_file "variance" calc.py` 找的是包含“variance”这个字符串的行。但代码的真正组织单位不是行，而是**类、方法、属性**。AutoCodeRover 用 AST（抽象语法树）解析代码，提供语义级的搜索 API。

#### AST 搜索 API

AutoCodeRover 为 Agent 提供了一组基于代码结构的搜索工具：

```
search_class("Calculator")           # 返回类定义及方法签名列表
search_method("stats_report")       # 返回方法的完整代码
search_method_in_class("median", "Statistics") # 精确到类中的方法
search_code("variance")             # 跨文件搜索代码片段
search_code_in_file("pass_rate", "calc.py") # 限定文件的代码搜索
```

与 SWE-agent 的文本搜索对比：

| 维度   | SWE-agent      | AutoCodeRover   |
|------|----------------|-----------------|
| 搜索粒度 | 文本行            | AST 节点（类/方法/属性） |
| 信息单位 | 文件片段（100 行窗口）  | 语义单元（一个完整方法）    |
| 导航方式 | 打开文件 → 滚动 → 搜索 | 直接跳到目标结构        |
| 隐喻   | 在书架上翻书找某一页     | 用目录直达章节         |

## 迭代上下文检索

AutoCodeRover 不是一次搜索就结束。它允许 Agent 多轮迭代地检索上下文：

```
Round 1: 根据 issue 描述，搜索相关类和方法  
         search_method("stats_report") → 拿到方法全文  
  
Round 2: Agent 分析代码，发现需要了解 sorted 函数的用法  
         search_code_in_file("sorted", "calc.py") → 缩小范围  
  
Round 3: Agent 判断"我已经收集到足够的上下文了"  
         → 输出 patch
```

每一轮搜索，Agent 都会在 Thought 中分析“我还缺什么信息？”——这是 OODA 循环中 Observe 和 Orient 的紧密配合。

## 关键数据

AutoCodeRover 在 SWE-bench Lite 上达到 **19%**，高于 SWE-agent 的 12.47%。结构化搜索的优势在复杂代码库中更为明显——当目标函数分散在多个类中时，AST 导航比文本搜索效率高得多。

## CC 的结构化导航：LSP 的微妙定位

**CC 源码实证**（`src/tools.ts`，行 224）：CC 实现了完整的 LSP 工具（`goToDefinition`、`findReferences`、`documentSymbol`、`workspaceSymbol`，共 9 个操作），但它**默认不启用**——需要设置环境变量 `ENABLE_LSP_TOOL=true` 才会暴露给模型。即使不启用 LSP Tool，LSP 基础设施仍会启动（通过插件加载语言服务器），用于**被动诊断**——当 Edit/Write 修改文件时，语言服务器会推送错误诊断，作为附件返回给模型。

这个设计选择值得玩味：CC 花了大量工程投入实现 LSP（861 行工具代码 + 完整的服务管理层），但默认不开放给模型主动使用。可能的原因：

1. **LSP 依赖外部语言服务器**（如 `typescript-language-server`），不是所有环境都安装了——可靠性不够
2. **Glob + Grep + Read 组合已经足够**，对 Opus/Sonnet 级别的模型来说，通用搜索 + 模型推理 > 专用结构化工具
3. **被动诊断已经提供了 AST 级信息的核心价值**——语法错误、类型错误自动推送，不需要模型主动查询



需要澄清一个概念：**LSP 不是 AST，而是一个协议层**。LSP（Language Server Protocol）是 JSON-RPC 通信协议，CC 通过它与外部语言服务器交互。底层的语言服务器（如 TypeScript 编译器）内部确实用 AST 做分析，但 CC 本身没有 AST 代码——它只发送/接收 LSP 消息。这与 AutoCodeRover 的方案有本质区别：AutoCodeRover 自己解析 AST 并提供搜索 API，CC 则把结构分析外包给语言服务器。

这体现了 CC 的设计选择：**让模型智能驱动搜索策略，而不是预设固定的搜索 API**。

AutoCodeRover 的思路是“给模型专用的 AST 搜索工具”——工具本身就包含了结构感知。CC 的思路是“给模型通用的搜索工具（Glob、Grep、Read）+ 被动 LSP 诊断”——结构感知由模型的推理能力来填补。两种方案的 trade-off：

|      | AutoCodeRover 方案 | CC 方案          |
|------|------------------|----------------|
| 工具智能 | 高（工具本身理解代码结构）    | 低（通用文本搜索）      |
| 模型智能 | 低（模型只需选对 API）    | 高（模型需自主决定搜索策略） |
| 灵活性  | 低（只能搜 AST 定义的结构） | 高（任何文本模式都能搜）   |
| 适用场景 | 结构清晰的 OOP 代码     | 任何代码（含脚本、配置文件） |

第二讲（§ 2.2）讨论 Action Space 时提到的“工具智能 vs 模型智能”的 trade-off 在这里再次出现。CC 选择了依赖强模型的智能来弥补工具的简单性——这对 Opus/Sonnet 级别的模型来说是合理的；但对本地 30B MoE 模型，更智能的工具（如 AST 搜索）可能是必要的补偿。

## 4.4 Agentless 的层次化定位：不需要 Agent 的 Localization

Agentless (Xia et al., UIUC, 2024) 提出了一个更激进的方案：**Localization 根本不需要 Agent 循环，分层缩小范围就够了**。

### 三层漏斗

Agentless 的 Localization 是一个固定的三层流水线，每层只做一次 LLM 调用：

第一层：文件级定位

输入：repo 文件树 + issue 描述

输出："可能需要修改的文件 Top-N"

关键：只给模型看文件树（文件名和目录结构），不看代码内容

第二层：类/函数级定位

输入：Top-N 文件的代码骨架（skeleton） — 类名、方法签名、不含方法体

输出："可能需要修改的类/函数"

关键：只给模型看"骨架"，不看完整代码

第三层：编辑位置定位

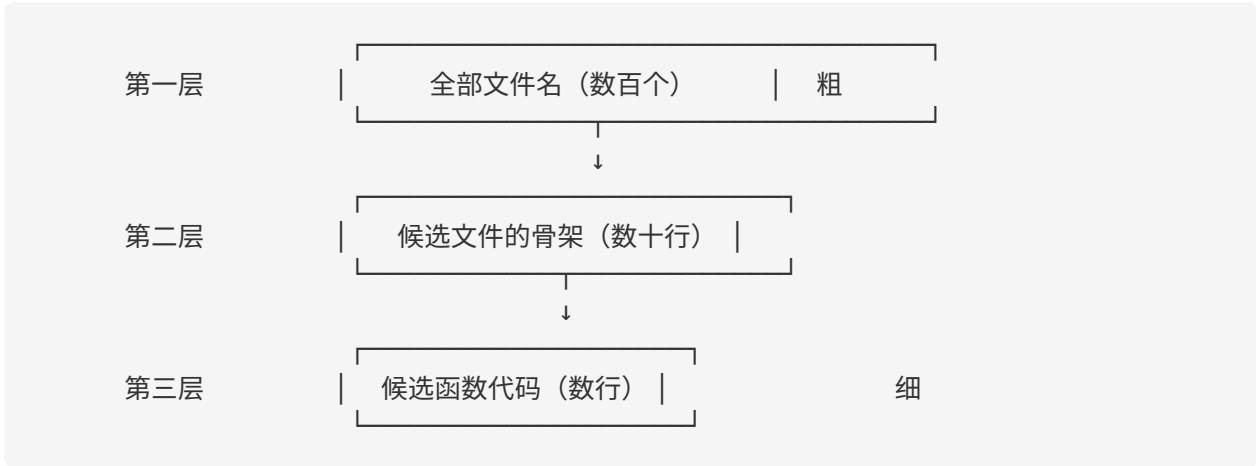
输入：候选函数的完整代码

输出："具体哪几行需要修改"

关键：到这一层才给完整代码，但范围已经很小了

每层都精心控制了**信息量**：

- 第一层：全仓库的文件树可能有几百行，但不含任何代码 → **最大覆盖，最小信息密度**
- 第二层：只看骨架（类名+方法签名），一个 500 行的文件被压缩为 20 行骨架 → **结构层面的信息**
- 第三层：只看候选函数的代码，可能只有 30 行 → **行级精度**



### 关键数据：成本与效率的颠覆

Agentless 论文 Table 1 提供了目前最完整的成本对比（SWE-bench Lite）：

| 方法                       | 解决率           | 平均成本/issue    | 平均 token/issue |
|--------------------------|---------------|---------------|----------------|
| SWE-agent (Claude 3.5 S) | 23.00%        | \$1.62        | 521,208        |
| SWE-agent (GPT-4)        | 18.33%        | \$2.53        | 498,346        |
| CodeR                    | 28.33%        | \$3.34        | 323,802        |
| AutoCodeRover            | 19.00%        | \$0.45        | 38,663         |
| <b>Agentless</b>         | <b>32.00%</b> | <b>\$0.70</b> | <b>78,166</b>  |

| 方法                  | 解决率   | 平均成本/issue | 平均 token/issue |
|---------------------|-------|------------|----------------|
| RAG + Claude 3 Opus | 4.33% | \$0.25     | —              |

几个关键发现：

- **SWE-agent 吞噬 token**：每个 issue 消耗 50 万 token，是 Agentless 的 **6-7 倍**。多轮交互式搜索的代价——Agent 每轮 Observe 都在消耗上下文空间
- **AutoCodeRover 最省 token**：仅 38,663 token/issue，得益于 AST 搜索直达目标，减少了冗余信息
- **Agentless 的性价比最高**：最高解决率 + 中等成本，因为三层漏斗每层精确控制信息量

Agentless 的成本进一步拆解（Table 2-4）：

|                    |        |          |
|--------------------|--------|----------|
| Localization（三层漏斗） | \$0.15 | ← 最便宜的阶段 |
| Repair（多候选采样）      | \$0.29 |          |
| Validation（测试筛选）   | \$0.26 |          |
|                    |        |          |
| 总计                 | \$0.70 |          |

**Localization 只占总成本的 21%**——因为三层漏斗每层只需一次 LLM 调用，且信息量严格控制。对比 SWE-agent 的 Localization 可能消耗总成本的 60%+（多轮搜索 + 文件浏览的 token 开销）。这是信息粒度控制带来的直接经济效益。

## 这挑战了什么认知？

Agentless 的成功提出了一个根本性问题：

如果固定流水线 > Agent 循环，那循环有什么用？

答案需要区分 Localization 和 Repair：

- **Localization（定位）的核心是信息筛选**——从大量信息中逐步缩小范围。这是一个“漏斗”操作，天然适合分层流水线：每层做一次决策，逐步缩小范围。不需要循环，不需要试错。
- **Repair（修复）和 Validation（验证）的核心是试错**——改了代码，跑测试，发现不对，再改。这是一个“循环”操作，需要反馈驱动。Agentless 在 Repair 阶段用多候选采样代替循环（生成 N 个 patch，选最好的），但**缺乏迭代修复能力**——它不能“试一下 → 发现不对 → 换个思路”。

所以 Agentless 的成功告诉我们：

**Observe 环节的核心能力是信息筛选（漏斗），不是推理循环。但 Orient 和 Act 环节仍然需要循环——这就是第三讲的领地。**

这也解释了为什么 Claude Code 的实际行为与 Agentless 的 Localization 高度同构：

| Agent 动态执行                 | Agentless 固定流水线 |
|----------------------------|-----------------|
| Glob "*.py" → 看到文件列表       | 第一层：看文件树        |
| Grep "median" → 定位候选文件     | 第二层：看代码骨架       |
| Read calc.py 0:50 → 读取关键代码 | 第三层：看具体代码       |

骨架相同，区别只是 Agent 在 ReAct 循环中动态决定搜索策略，而 Agentless 预设了固定的三步流水线。Agent 的优势在复杂场景中才显现：当第一次搜索方向错误时，Agent 能根据结果调整策略（“这个文件不对，换一个方向”），而 Agentless 的流水线不可回溯。

## 4.5 [Demo 4 · Live] 在真实代码库中观察 Localization

前几讲的 Demo 都在 39 行的 `buggy_calc.py` 上操作——Agent 一次 Read 就看到全貌。现在我们把同一个 30B MoE Agent 放进一个真实的大型项目中，看看 Localization 策略会发生什么变化。

### 任务：在 CC 源码中定位 Edit 工具的校验逻辑

Claude Code 开源代码库（`claude-code/`）有 **1902 个 TypeScript 文件**。给 Agent 的任务是：

“claude-code 的 Edit 工具在什么情况下会拒绝执行？找到相关的校验逻辑。”

Agent 的实际行为轨迹：

|          |                                                                |             |
|----------|----------------------------------------------------------------|-------------|
| Round 1: | Glob("**/*Edit*") → 找到 <code>src/tools/FileEditTool/</code> 目录 | 文件级定位       |
| Round 2: | Grep("validate reject", path="FileEditTool.ts")                | 函数级定位       |
| Round 3: | Read("FileEditTool.ts", offset=130, limit=200)                 | 行级定位        |
| Round 4: | [Thought] "找到了： <code>old_string</code> 匹配多处时拒绝执行"             | Orient：理解逻辑 |

注意这个过程和 Agentless 三层漏斗的对应关系——Agent 在 ReAct 循环中动态执行了几乎相同的层次化策略：文件级 → 函数级 → 行级 → 理解。

在这个过程中有一个值得关注的细节：当 Agent 试图直接 Read `FileEditTool.ts`（625 行）时，触发了 Read 门槛——工具返回 Error + Grep 建议。Agent 随后改用 Grep 精确定位到 `validateInput` 方法，再用 `offset/limit` 只读取需要的部分。这正是 ACI 原则三（信息丰富的反馈）在起作用：**错误信息本身就是教学信号，引导 Agent 切换到更紧凑的 Observe 策略。**

这个门槛在 Demo 前一版中不存在，导致 Agent 读入大文件后上下文溢出，vLLM 返回空响应，整个会话崩溃（“No response from server”）。加入门槛后，Agent 至少能收到有用的错误反馈而不是静默失败——**ACI 护栏把“不可恢复的系统故障”变成了“可诊断的工具错误”。**

### 对比：小文件 vs 大项目

回顾第三讲 Demo 中 30B 修复 `buggy_calc.py` 的过程（1 轮 Read 直接搞定），对比大项目定位：

| 维度              | <code>buggy_calc.py</code> （39 行） | CC 源码（1902 文件）                                |
|-----------------|-----------------------------------|-----------------------------------------------|
| Localization 轮数 | 1 轮（直接 Read）                      | 4-5 轮（搜索→缩小→精读）                               |
| 信息控制            | 不需要                               | 必须（用 Grep 缩小、Read <code>offset/limit</code> ） |
| 策略              | 无漏斗（一步到位）                         | 类 Agentless 三层漏斗                              |
| token 消耗        | ~200 token                        | ~3000-5000 token                              |

Agent 用了 5 轮才找到目标，而 Agentless 用三层固定流水线、三次 LLM 调用就能做到同样的事。哪个更好？这取决于任务复杂度——简单定位适合漏斗，需要动态调整方向时 Agent 循环才有优势。

## 4.6 三种 Localization 策略对比

### 总表

| 维度 | SWE-agent         | AutoCodeRover  | Agentless |
|----|-------------------|----------------|-----------|
| 方法 | ACI 工具 + Agent 循环 | AST API + 迭代检索 | 三层漏斗（无循环） |

| 维度             | SWE-agent | AutoCodeRover | Agentless       |
|----------------|-----------|---------------|-----------------|
| 搜索粒度           | 文本行       | AST 节点（类/方法）  | 层次递进（文件→函数→行）   |
| 信息控制           | 100 行窗口   | 语义单元          | 骨架 → 全文         |
| SWE-bench Lite | 12.47%    | 19%           | 32%             |
| 成本             | 高（多轮交互）   | 中             | 低（\$0.70/issue） |
| 优势             | 通用、灵活     | 结构感知          | 高效、便宜           |
| 劣势             | token 消耗大 | 依赖 AST 解析     | 无迭代修复能力         |

### 三种策略的信息控制隐喻

SWE-agent：拿着手电筒在黑暗仓库里搜索（每次照亮一小片区域）  
AutoCodeRover：拿着仓库的货架目录直接走到对应货架  
Agentless：先看仓库平面图 → 再看某排货架的标签 → 最后打开具体箱子

三种方法都在解决同一个问题：**在上下文窗口的约束下最大化信息密度**。只是搜索策略不同——手电筒式扫描（SWE-agent）、结构化跳转（AutoCodeRover）、分层缩小（Agentless）。

### MVP 当前工具与论文工具的映射

| 论文工具                          | MVP 对应               | CC 对应                        | 差距分析             |
|-------------------------------|----------------------|------------------------------|------------------|
| SWE-agent<br>find_file        | Glob 工具              | Glob 工具                      | 功能对等             |
| SWE-agent<br>search_dir       | Grep                 | Grep                         | 功能对等             |
| SWE-agent File Viewer         | Read（有 offset/limit） | Read（有 offset/limit）         | 基本对等             |
| AutoCodeRover<br>search_class | 无                    | LSP<br>documentSymbol（默认未启用） | MVP 缺少结构化搜索      |
| Agentless 文件树                 |                      | Glob + 文件树快照                 | MVP 已实现自适应文件树（小项 |

| 论文工具           | MVP 对应              | CC 对应     | 差距分析                |
|----------------|---------------------|-----------|---------------------|
|                | system prompt 文件树快照 |           | 目显示完整文件，大项目只显示目录结构) |
| Agentless 代码骨架 | 无                   | 无（模型自行提取） | 两者都依赖模型智能           |

MVP 的 Localization 工具集已基本覆盖 SWE-agent 的工具栈（Glob + Grep + Read），并在 system prompt 中嵌入了项目文件树快照（对应 Agentless 第一层）。仍缺少的是 AutoCodeRover 的结构化搜索能力（AST/LSP 级别），以及 Agentless 的代码骨架提取。进一步改进的方向：

- 1. 增加 **Tree** 工具（结构化文件浏览）——给模型提供 Agentless 第二层所需的代码骨架视图
- 2. 对 system prompt 文件树做更智能的深度控制——当前大项目只展示前两层目录

## 4.7 本节小结：信息粒度是 Observe 的核心战场

回到二维认知地图，本节覆盖了 Observe 环节：

|                 | Localization                                             | Repair | Validation |
|-----------------|----------------------------------------------------------|--------|------------|
| Observe<br>(本节) | ★ SWE-agent ACI<br>★ AutoCodeRover AST<br>★ Agentless 漏斗 |        |            |
| Orient          | (第三讲)                                                    |        |            |
| Act             | (第二讲)                                                    |        |            |

三个核心收获：

### 1. Observe 质量 = 信息粒度控制

不是给 Agent 更多信息，而是给“刚好够”的信息。SWE-agent 的 100 行窗口、AutoCodeRover 的方法级语义单元、Agentless 的骨架→全文层次递进，都是在解决同一个问题：**在上下文窗口的约束下最大化信息密度**。CC 的 Read offset/limit、Grep head\_limit、Tool result budget 是同一思路的工业级实现。

## 2. ACI > Model

SWE-agent 的核心发现——改善工具接口设计（ACI）带来的提升（+10.7pp）远大于换更好的模型。这是 Boyd 推论二的实证：**感知设计决定循环质量上限**。对 MVP 的启示：与其追求更大的模型，不如优化工具的输出格式。

## 3. Localization 不一定需要循环

Agentless 用三层固定流水线达到了最高的定位准确率（文件级 Top-5: 79.4%），成本仅 \$0.70。这说明 Observe 环节的核心能力是**信息筛选（漏斗）**，不是试错循环。但 Repair 和 Validation 仍需要循环——Agentless 的定位很准，但它不能从失败的修复中学习，这正是第三讲 Reflexion 要解决的问题。

---

下一节的问题是：本节展示了 Agentless 的三层漏斗在 Localization 上超越了 Agent 方案，成本仅 \$0.70。如果固定流水线在定位阶段就能击败 Agent 循环，那整个 L→R→V 流程都用流水线行不行？修复一个 Bug，一定需要完整的 Agent 循环吗？